

API DEVELOPER — APIGEE / AWS

Apigee + AWS

Interview Preparation Guide

From GCP Apigee expertise to AWS-backed API platforms — architecture,
token flow,
routing, security, scenarios and a full mock Q&A bank

Architecture & Component Deep-Dive

Login / Token / Routing Flows

OAuth 2.0 & MTLS

6 Real-World Scenarios

45+ Interview Q&A

3-Day Study Plan

Prepared for: Sai Rathish | Prodapt-AID Practice | L&T Technology Services — Apigee Developer role | August 2026

Table of Contents

- 1 How to use this guide + 3-Day Study Plan
- 2 GCP Apigee → AWS: Concept Map for a Fast Ramp-Up
- 3 Reference Architecture Diagram
- 4 Component Specifications & Functionality
- 5 End-to-End Request Flow — Where It Starts and Ends
- 6 Deep Dive: Customer Login & Token Generation
- 7 Deep Dive: How Apigee Validates the Token
- 8 Deep Dive: Routing to Multiple Backend Microservices
- 9 Apigee Policy Cheat-Sheet (Caching, AccessEntity, RaiseFault & more)
- 10 Security Deep Dive: OAuth 2.0 Grant Types & MTLS
- 11 AWS Services Reference (mapped to the JD)
- 12 Zero-Downtime Deployment & Schema Refactoring
- 13 CI/CD Tooling: Git, Jenkins, Sonar, Splunk, Maven
- 14 SLO / SLA / SLI, On-Call & Blameless Postmortems
- 15 Real-World Scenarios
- 16 Mock Interview Q&A Bank
- 17 Final Recall Checklist

1. How to Use This Guide + 3-Day Study Plan

You already know Apigee deeply from the GCP side — policies, proxies, OAuth flows and API lifecycle management are the same discipline regardless of cloud. What is new to you is the **AWS vocabulary** this JD leans on: the specific services (EC2, VPC, RDS, Lambda, ECS, CloudFormation, IAM, ACM, ELB, EBS, CloudFront/WAF, SQS/SNS/SES, DynamoDB) and how they sit "underneath" or "beside" an Apigee proxy that fronts them. The fastest way to close that gap in three days is not to learn AWS end-to-end — it's to learn exactly how each of those services plays a role in the one architecture this document walks through, and to be able to narrate the request flow through it fluently.

Day 1 — Architecture & Flow (today)

- Read Sections 2-8 once, slowly. Trace the numbered arrows in the diagram (Section 3) with your finger while reading the End-to-End Flow (Section 5) and the Login/Token/Routing deep-dives (Sections 6-8).
- Redraw the diagram from memory on paper, labeling all 10 steps. Do this twice.
- Say out loud, in your own words: "a client logs in, gets a token, Apigee validates it, and routes it to the right microservice" — then add one sentence per AWS service explaining where it sits in that sentence.

Day 2 — Policies, Security & AWS Services

- Study Section 9 (policy cheat-sheet) — you know these concepts already from Apigee Edge/GCP; just anchor each policy to a one-line "why/when" so you can answer fast.
- Study Section 10 (OAuth grant types + MTLS) until you can draw the MTLS handshake (client ↔ Apigee and Apigee ↔ backend) from memory.
- Study Section 11 (AWS services reference) — for each service, memorize: (a) one-line purpose, (b) its GCP equivalent, (c) where it appears in the architecture diagram.
- Read Section 12 (zero-downtime) and Section 13 (CI/CD tooling) — these map directly to explicit JD bullets.

Day 3 — Scenarios, Mock Q&A & Final Polish

- Work through all 6 scenarios in Section 15 out loud, as if answering a "tell me about a time..." interview question.
- Do a full pass of the Q&A bank in Section 16. Cover the answer, read only the question, and answer from memory before checking.
- Finish with the one-page recall checklist in Section 17 the morning of the interview.

Interview framing tip: When asked an AWS question you're less sure of, it is completely fair — and often impressive — to answer by anchoring to your GCP experience: *"In GCP I'd use Cloud Armor / Cloud Endpoints / Cloud Functions for this; on AWS the equivalent is WAF / API Gateway / Lambda, and here's how I understand it fits..."* This shows transferable engineering judgment rather than rote memorization, which is exactly what a "strong in basics" hiring bar is testing for.

2. GCP Apigee → AWS: Concept Map for a Fast Ramp-Up

Apigee itself does not change based on which cloud hosts your backends — the proxy engine, policies, analytics and developer portal work identically whether you are on Apigee Edge, Apigee X, or Apigee hybrid, and regardless of whether the API's *target* (backend) lives in GCP, AWS, on-prem, or a mix. What changes is everything *around* Apigee: the network, identity, compute and data services that the backend and the surrounding platform rely on. The table below maps each AWS service in the JD to the closest GCP concept you already know, so you can reason by analogy instead of memorizing from zero.

AWS Service (from JD)	Closest GCP Equivalent	One-line purpose
EC2	Compute Engine (VM instances)	Virtual machines for compute workloads; here, hosts a legacy/monolithic microservice behind an Auto Scaling Group.
ECS (Fargate/EC2 launch type)	Cloud Run / GKE	Runs containerized microservices without (Fargate) or with (EC2 launch type) managing the underlying hosts.
S3	Cloud Storage	Object storage — Swagger/OpenAPI specs, build artifacts, logs, static assets.
VPC	VPC Network	Isolated private network with subnets, route tables, security groups (~firewall rules) around all workloads.
Route 53	Cloud DNS	DNS resolution for domains and internal service discovery; also does health-check based failover routing.
RDS	Cloud SQL	Managed relational database (MySQL/PostgreSQL/etc.) for structured, transactional data.
DynamoDB	Firestore / Bigtable	Managed NoSQL key-value/document store — low-latency lookups (sessions, preferences, idempotency keys).
CloudFormation	Deployment Manager / Terraform-on-GCP	Infrastructure-as-code: declarative templates that provision the whole stack repeatably.
Lambda	Cloud Functions	Serverless, event-driven compute — used for lightweight microservices, glue code, and async consumers.
CloudWatch (logging)	Cloud Logging / Cloud Monitoring	Centralized logs, metrics, dashboards and alarms across every AWS resource.
IAM	Cloud IAM	Identity & access management — roles/policies controlling who/what can call which AWS resource.
Certificate Manager (ACM)	Google-managed SSL certificates	Issues and auto-renews TLS certificates used by ALB/CloudFront/API Gateway.

ELB (ALB/NLB)	Cloud Load Balancing	Distributes traffic across healthy backend instances/containers; ALB = L7 (HTTP), NLB = L4 (TCP).
EBS	Persistent Disk	Block storage volumes attached to EC2 instances.
CloudFront + WAF	Cloud CDN + Cloud Armor	Edge CDN caching plus a Web Application Firewall filtering malicious/abusive traffic before it reaches origin.
SQS	Pub/Sub (queue-like usage) / Cloud Tasks	Managed message queue that decouples producer and consumer microservices.
SNS	Pub/Sub (topic/fan-out)	Pub/sub notification service — one event fans out to many subscribers (email, SQS, Lambda).
SES	—	Transactional email sending service (order confirmations, OTPs, alerts).

Interview soundbite: "Apigee is cloud-agnostic by design — the same ProxyEndpoint/TargetEndpoint model, the same policy set (VerifyAPIKey, OAuthV2, Quota, SpikeArrest, RaiseFault, Caching) works whether the backend sits on GCP, AWS, or on-prem. What I need to layer in for AWS is how the *target* side — networking, TLS, compute and observability — is composed using VPC, ALB/NLB, ACM, ECS/Lambda/EC2 and CloudWatch."

3. Reference Architecture Diagram

This is the architecture referenced throughout the rest of this guide: a client application, an OAuth token endpoint, an Apigee proxy layer, and an AWS backend hosting three microservices behind a load balancer, with supporting data, messaging and cross-cutting (security/observability/IaC) services. The numbers on the diagram correspond exactly to the numbered steps used in Section 5 (End-to-End Flow) — learn this diagram once and every later section becomes easy to place.

Apigee + AWS Reference Architecture — End-to-End Request & Token Flow

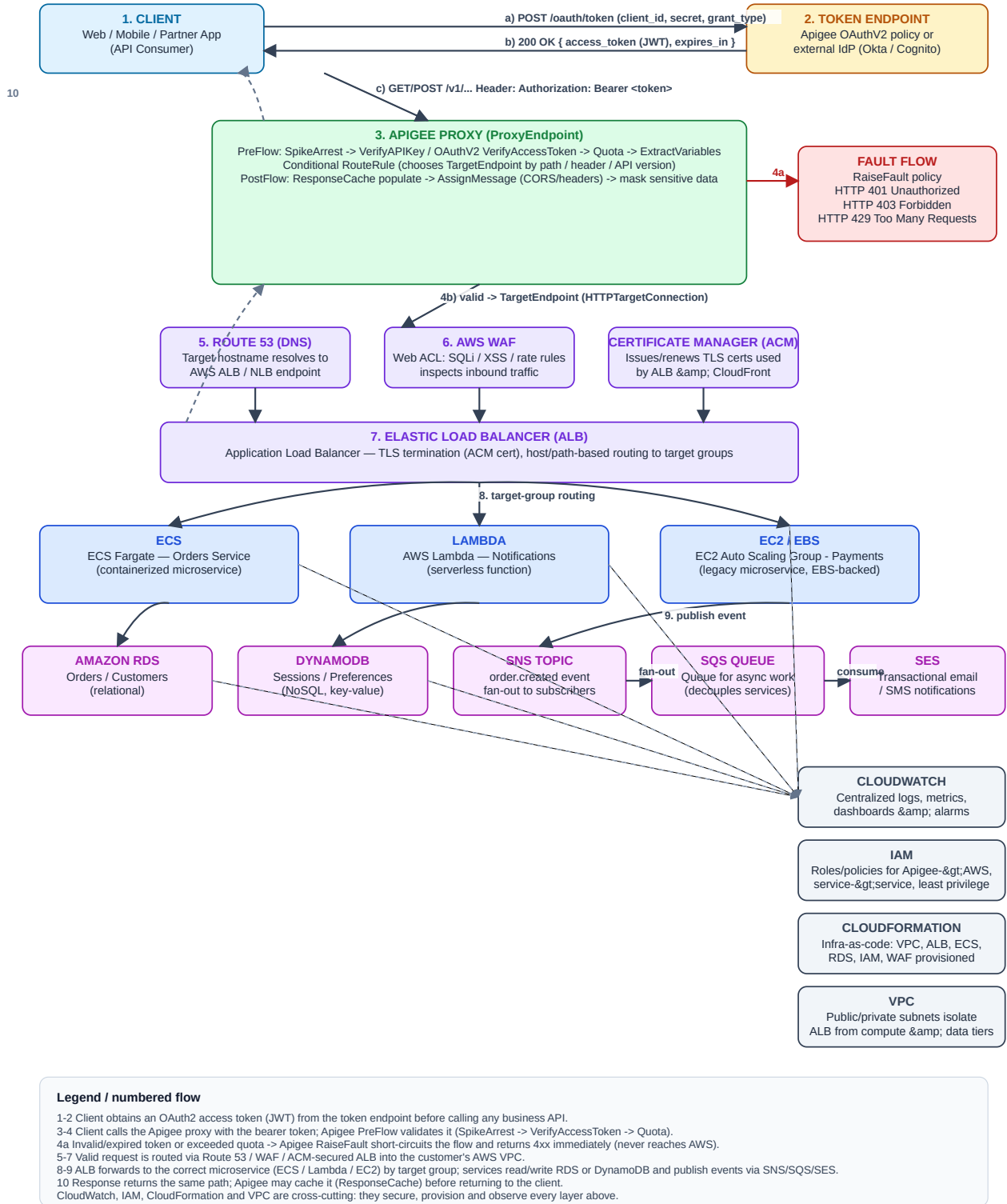


Figure 1 — Apigee + AWS reference architecture used throughout this guide (also see Sections 6-8 for the login/token, validation and routing deep-dives).

4. Component Specifications & Functionality

Below is a specification-level description of every box in the diagram: what it is, its core configuration, and the specific job it does in this flow. Treat this as the "what does X actually do here" answer key.

4.1 Client-facing / Identity layer

Component	Specification	Functionality in this flow
Client App	Web SPA, mobile app, or partner server-to-server integration; holds a client_id/secret or redirects a user through a login page.	Initiates the token request, then attaches the resulting token to every subsequent API call as an Authorization: Bearer header.
Token Endpoint (Apigee OAuthV2 policy, or external IdP such as Okta/Cognito/Azure AD)	Apigee /oauth/token flow implemented via the OAuthV2 policy (GenerateAccessToken operation), or a federated external IdP that Apigee trusts via JWKS.	Authenticates the caller's credentials, issues a signed access token (JWT or opaque reference token) plus optional refresh token, and defines the token's scopes/expiry.

4.2 Apigee layer

Component	Specification	Functionality in this flow
ProxyEndpoint	Defines the externally exposed base path (e.g. /v1/orders) and the HTTP(S) listener config (VirtualHost / TLS).	Entry point for every inbound client call; hosts the PreFlow, Flow (conditional), and PostFlow policy chains.
PreFlow policies	SpikeArrest → VerifyAPIKey/OAuthV2 (VerifyAccessToken) → Quota → ExtractVariables.	Rate-limits abusive bursts, authenticates & authorizes the token, enforces per-app/per-plan quotas, and pulls values (claims, headers, query params) into flow variables for later use.
Conditional RouteRule	XML/YAML condition blocks evaluating path suffix, header, or query param (e.g. request.header.x-api-version = "2").	Chooses which TargetEndpoint (i.e., which backend microservice) handles this specific request — this is the routing decision point.
TargetEndpoint / HTTPTargetConnection	Points to a backend URL (an AWS Route 53 DNS name resolving to an ALB/NLB), with optional SSLInfo block for mTLS client-cert presentation.	Forwards the validated request into AWS; receives the backend's response and returns it up the chain.
PostFlow policies	ResponseCache (populate), AssignMessage (headers/CORS), mask/redact policies.	Optionally caches cacheable responses, normalizes headers, strips

		sensitive fields before returning to the client.
Fault Flow / RaiseFault	Triggered by any PreFlow or Flow failure condition.	Short-circuits the request and returns a structured 4xx (401/403/429) without ever reaching AWS — this is a critical cost/security control.

4.3 AWS network & edge layer

Component	Specification	Functionality in this flow
Route 53	Hosted zone with A/CNAME/ALIAS records; supports health-check based failover and latency-based routing.	Resolves the backend hostname that Apigee's TargetEndpoint calls to the current ALB/NLB IP.
AWS WAF	Web ACL with managed + custom rules (SQLi, XSS, rate-based rules, IP reputation lists), attached to ALB/CloudFront.	Inspects and blocks malicious requests before they reach compute — a second line of defense behind Apigee's own policies.
Certificate Manager (ACM)	Public/private TLS certificates, auto-renewed, region-scoped.	Supplies the TLS certificate the ALB/CloudFront presents to Apigee (and to end users if Apigee isn't the sole edge).
Elastic Load Balancer (ALB)	Layer-7 load balancer with listener rules for host/path-based routing to distinct target groups; health checks per target group.	The single AWS-side entry point Apigee's TargetEndpoint talks to; fans requests out to the correct microservice's target group.
VPC	Custom CIDR range, public subnets (ALB, NAT gateway) and private subnets (compute, data tier), security groups and NACLs.	Network isolation boundary — only the ALB is internet/Apigee-reachable; compute and data tiers sit in private subnets.

4.4 Compute (microservices) layer

Component	Specification	Functionality in this flow
ECS (Fargate)	Task definitions (container image, CPU/memory), Service with desired count, registered to an ALB target group.	Runs the "Orders" microservice as containers with no host management; scales the task count based on load.
Lambda	Function with a handler, memory/timeout config, triggered via ALB target group (Lambda target type) or an API Gateway/SNS/SQS event source.	Runs the lightweight "Notifications" microservice — scales to zero, pay-per-invocation.
EC2 Auto Scaling Group	Launch template, min/max/desired capacity, scaling policies (target tracking on CPU/requests), backed by EBS volumes.	Runs a legacy "Payments" microservice not yet containerized; scales horizontally by adding/removing instances.

4.5 Data, messaging & cross-cutting layer

Component	Specification	Functionality in this flow
RDS	Managed relational engine (e.g. PostgreSQL), Multi-AZ for HA, automated backups.	System of record for structured, transactional data (orders, customers).
DynamoDB	On-demand or provisioned capacity, partition/sort key design, optional TTL and streams.	Low-latency key-value store for session state and preferences the Notifications service reads.
SNS	Topic with one or more subscriptions (SQS, Lambda, email, SMS).	Publishes a domain event (e.g. <code>order.created</code>) once, fanning it out to every interested consumer.
SQS	Standard or FIFO queue, visibility timeout, dead-letter queue (DLQ) for poison messages.	Buffers work for asynchronous consumers, decoupling producer and consumer availability/scaling.
SES	Verified sending domain, configuration sets, bounce/complaint handling.	Sends transactional email (order confirmation) triggered by a queue consumer.
CloudWatch	Log groups, custom metrics, dashboards, alarms wired to SNS for paging.	Central place engineers (and Apigee's own Analytics/Splunk integration) use to observe latency, error rate and saturation — feeds SLO/SLA dashboards and on-call alerts.
IAM	Roles with least-privilege policies; ECS task roles, Lambda execution roles, cross-account trust where needed.	Controls exactly which AWS resource each service (and Apigee's service-callouts, if any) is permitted to touch.
CloudFormation	Nested stacks/templates version-controlled in Git, deployed via CI/CD pipeline.	Provisions the entire AWS footprint (VPC, ALB, ECS, RDS, IAM, WAF) repeatably across dev/stage/prod.

5. End-to-End Request Flow — Where It Starts and Ends

The flow always starts at the client and always ends either (a) back at the client with a response, or (b) short-circuited at Apigee with a fault — it never "ends" inside AWS; AWS is a middle hop, not the destination. Walk through it exactly in this order when explaining it in an interview.

1. **Token acquisition (starts here).** The client calls the token endpoint (Apigee's own `OAuthV2` policy exposed at `/oauth/token`, or an external IdP) with a grant type appropriate to its use case — `client_credentials` for server-to-server, `authorization_code` for a user-facing login, or `password` / `refresh_token` for legacy or renewal flows.
2. **Token issuance.** The token endpoint validates the credentials, mints a signed JWT (or opaque token) with claims (subject, scopes, expiry, audience) and returns it to the client. This step never touches AWS at all.
3. **Business API call.** The client calls the actual Apigee ProxyEndpoint (e.g. `POST https://api.company.com/v1/orders`) with `Authorization: Bearer <token>`.
4. **PreFlow — SpikeArrest.** Apigee immediately smooths traffic bursts (e.g. max 30 requests/second) to protect both Apigee and the eventual AWS backend from being overwhelmed.
5. **PreFlow — VerifyAPIKey / OAuthV2 VerifyAccessToken.** Apigee checks the token's signature (against a cached JWKS/verification key), expiry, and required scopes. This is a local, in-Apigee check — no AWS call is made yet.
6. **PreFlow — Quota.** Apigee checks the calling app hasn't exceeded its plan's allowed call volume (e.g. 10,000 calls/day).
7. **Decision point.** If any PreFlow check fails, **RaiseFault fires and the flow ends right here** with a 401/403/429 — AWS is never invoked. This is the answer to "where can the flow end early."
8. **Conditional routing.** If all checks pass, a RouteRule inspects the path/header/version and selects the correct TargetEndpoint — this is the moment the request is "assigned" to a specific microservice (Orders, Notifications, or Payments in our diagram).
9. **Egress to AWS.** The chosen TargetEndpoint's HTTPTargetConnection calls a hostname that Route 53 resolves to an ALB, guarded by WAF, TLS-terminated using an ACM certificate.
10. **ALB routing.** The ALB's listener rules match the path/host to a target group and forwards to the healthy ECS task, Lambda function, or EC2 instance behind it.
11. **Backend processing.** The microservice executes business logic, reading/writing RDS or DynamoDB as needed, and optionally publishing an event to SNS (which may fan out to SQS → another consumer → SES for a notification email).
12. **Response return (ends here).** The microservice's response travels back through the ALB → Apigee's TargetEndpoint → PostFlow (optionally cached via ResponseCache, headers normalized) → and finally back to the client as the HTTP response. This is the normal end of the flow.

Interview one-liner: "The flow starts when the client requests a token, and ends either when Apigee raises a fault before ever reaching AWS, or when the response from the microservice —"

after touching RDS/DynamoDB/SNS as needed — bubbles back through the ALB and Apigee to the client."

6. Deep Dive: Customer Login & Token Generation

"How does a customer log in and get a token" almost always maps to one of two patterns. Know both, and know which one fits which client type.

6.1 Pattern A — Server-to-server / trusted client (client_credentials)

Used when the caller is not a human but a system (a partner's backend, a batch job). There is no "login screen" — the client authenticates itself directly.

```
POST /oauth/token HTTP/1.1
Host: api.company.com
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&client_id=abc123&client_secret=s3cr3t
```

Apigee's `OAuthV2` policy (GenerateAccessToken operation) verifies the client_id/secret against the registered Developer App in Apigee's own store, then mints and returns a token:

```
{
  "access_token": "eyJhbGciOiJIUzUzIiwiaXNjaWQiOiJkZWVudC1234567890",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "orders.read orders.write"
}
```

6.2 Pattern B — End-user login (authorization_code + PKCE)

Used when an actual customer logs in through a UI (web/mobile app). This is the OAuth 2.0 Authorization Code flow, ideally with PKCE for public clients (mobile/SPA):

1. The app redirects the customer's browser to the identity provider's `/authorize` endpoint (could be Apigee acting as the authorization server, or a federated IdP like Cognito/Okta that Apigee trusts).
2. The customer enters credentials (and completes MFA if configured) directly on the IdP's login page — the client app never sees the raw password.
3. On success, the IdP redirects back to the app with a short-lived `authorization_code`.
4. The app (or its backend, to keep secrets off the device) exchanges that code for tokens at `/oauth/token`, presenting a PKCE `code_verifier` that must match the `code_challenge` sent in step 1.
5. The token endpoint returns an `access_token` (short-lived, e.g. 1 hour), a `refresh_token` (long-lived), and often an `id_token` (OpenID Connect, containing the user's identity claims).

The access token is a JWT with three parts — header, payload (claims), signature — for example:

```
Header: {"alg":"RS256","typ":"JWT","kid":"key-2026-01"}
Payload: {"sub":"user-8891","scope":"orders.read","aud":"api.company.com",
```

```
"iss":"https://api.company.com/oauth","exp":1755000000,"iat":1754996400}
Signature: RSASHA256(base64(header)+"."+base64(payload), private_key)
```

When the access token expires, the client uses the refresh token (`grant_type=refresh_token`) to get a new access token silently, without asking the customer to log in again — this is what "the customer stays logged in" means in practice.

Grant type	Used by	Credentials presented	Typical token lifetime
client_credentials	Server-to-server, partner integrations	client_id + client_secret	~1 hour, no refresh token (re-request when expired)
authorization_code (+PKCE)	Web/mobile apps with a real user	User login at IdP + authorization code + code_verifier	Access token ~1 hour; refresh token days/weeks
refresh_token	Any client renewing an expired access token	The previously issued refresh_token	New access token; refresh token may be rotated
password (legacy, avoid if possible)	First-party legacy apps only	Username + password directly to token endpoint	Same as authorization_code, but no delegated consent step

7. Deep Dive: How Apigee Validates the Token

This is one of the most commonly asked "explain the mechanics" questions. There are two viable validation strategies, and knowing the trade-off between them is what separates a basic answer from a strong one.

7.1 Strategy 1 — Local (stateless) validation of a JWT

If the token is a signed JWT, Apigee's `VerifyJWT` (or `OAuthV2 VerifyAccessToken` when Apigee itself issued it) policy can validate it entirely in-memory, with zero network round-trip on the hot path:

1. **Signature check.** Apigee fetches the issuer's public key — either from its own local Keystore (if Apigee is the issuer) or via the IdP's JWKS endpoint, cached for a TTL (commonly 1 hour) so most requests don't refetch it. It recomputes the signature over the header+payload and compares.
2. **Standard claims check.** `exp` (not expired), `nbf` (not-before, if present), `iss` (expected issuer), `aud` (expected audience — i.e., this API).
3. **Scope/role check.** A follow-up policy (often an `AccessEntity` or a JavaScript/condition policy) checks the token's `scope` claim contains what this operation requires (e.g. `orders.write` for a POST).
4. **Outcome.** Any failure triggers a fault variable that a `RaiseFault` policy turns into a 401 (invalid/expired) or 403 (valid but insufficient scope) response — before the request ever reaches the `RouteRule`, let alone AWS.

7.2 Strategy 2 — Introspection (stateful) validation of an opaque token

If the token is an opaque reference string (not a self-contained JWT), Apigee cannot validate it locally. Instead, a `ServiceCallout` policy calls the issuing IdP's `/introspect` endpoint (RFC 7662) synchronously, passing the token, and the IdP responds with `{"active": true, "scope": "...", "exp": ...}`. This adds latency (an extra network hop per request) but lets the IdP revoke a token immediately — with local JWT validation, a revoked-but-not-yet-expired JWT will still pass until it naturally expires, unless you also maintain a revocation/deny-list.

	Local JWT validation	Introspection (opaque token)
Latency	Very low (no network call on hot path)	Higher (extra call to IdP per request, though cacheable briefly)
Immediate revocation	Not by default — needs a deny-list/short TTL	Yes — IdP is authoritative on every check
Scales without IdP load	Yes	No — IdP is in the critical path
Apigee policy used	<code>VerifyJWT</code> / <code>OAuthV2 VerifyAccessToken</code>	<code>ServiceCallout</code> to introspection endpoint

How this connects to caching: Apigee commonly uses a `PopulateCache` / `LookupCache` pair (or the built-in JWKS cache) to avoid refetching the public key or re-introspecting the same token on every single call within a short window — this is a very likely follow-up interview question (see Section 9).

8. Deep Dive: Routing to Multiple Backend Microservices

Routing happens at *two* layers, and interviewers like candidates who distinguish them clearly: Apigee decides *which microservice*, and the AWS ALB decides *which healthy instance of that microservice*.

8.1 Layer 1 — Apigee conditional routing (which microservice)

A single Apigee ProxyEndpoint can front many backends. A RouteRule with conditions picks the TargetEndpoint:

```
<RouteRule name="OrdersRoute">
  <Condition>request.uri.path MatchesPath "/v1/orders/**"</Condition>
  <TargetEndpoint>OrdersTarget</TargetEndpoint>
</RouteRule>
<RouteRule name="NotifyRoute">
  <Condition>request.uri.path MatchesPath "/v1/notify/**"</Condition>
  <TargetEndpoint>NotifyTarget</TargetEndpoint>
</RouteRule>
<RouteRule name="PaymentsRoute">
  <Condition>request.uri.path MatchesPath "/v1/payments/**"</Condition>
  <TargetEndpoint>PaymentsTarget</TargetEndpoint>
</RouteRule>
```

Each TargetEndpoint's `HTTPTargetConnection` points at a distinct AWS-side hostname (e.g. `orders.internal.company.com`, `notify.internal.company.com`), each resolved by Route 53 to its own ALB listener/target group. This is also where you'd configure per-target load-balancing across multiple backend "servers" defined in Apigee (for weighted/canary rollout across versions — see Section 12), mTLS keystores, and connection timeouts.

8.2 Layer 2 — AWS ALB routing (which healthy instance)

The ALB owns a set of listener rules (host- and path-based) that map to target groups; each target group runs its own health checks (HTTP 200 on `/health`, say, every 30s) and only forwards to targets currently passing:

Listener rule	Target group	Compute type	Health check
Host: orders.internal.* → any path	orders-tg	ECS Fargate service (N tasks)	GET /health every 30s, 2 healthy threshold
Host: notify.internal.* → any path	notify-tg	Lambda function (Lambda target type)	Lambda's own success/failure response
Host: payments.internal.* → any path	payments-tg	EC2 Auto Scaling Group	GET /health every 30s via instance ENI

8.3 Why two layers instead of one

- **Separation of concerns.** Apigee's routing decision is about the API contract (versioning, path structure, which product/team owns this operation) — it doesn't need to know or care how many EC2 instances are behind Payments today.
- **Independent scaling.** AWS Auto Scaling / ECS service scaling can add or remove capacity without touching a single Apigee proxy bundle.
- **Zero-downtime deploys.** A rolling deploy of the Orders service only ever changes membership of `orders-tg` — Apigee's RouteRule and TargetEndpoint hostname never change, so client-visible contracts stay completely stable (see Section 12).
- **Multi-cloud/portability.** If Payments moved off AWS tomorrow, only its TargetEndpoint's hostname changes in Apigee — the rest of the proxy, and every client, is unaffected.

9. Apigee Policy Cheat-Sheet

The JD explicitly calls out caching, access entity and raise fault policies, plus general "Apigee policies" fluency. This table is a fast-recall reference — you already know these from GCP Apigee; the point here is to be able to state the "why/when" in one breath.

Policy	Type	Why / when you use it
VerifyAPIKey	Security	Simple app identification via an API key (no user context) — typical for public/free-tier APIs or as a first gate before OAuth.
OAuthV2 (GenerateAccessToken / VerifyAccessToken)	Security	Issues and validates OAuth 2.0 tokens when Apigee itself is the authorization server.
VerifyJWT / DecodeJWT	Security	Validates (or just decodes, without validating) a JWT issued by an external IdP — signature, expiry, issuer, audience.
SpikeArrest	Traffic management	Smooths sudden traffic bursts (e.g. max 10 requests/second) to protect the backend from being overwhelmed — different from Quota, which limits total volume over a longer window.
Quota	Traffic management	Enforces a subscription/plan limit (e.g. 10,000 calls/day) per app, per developer, or per API product.
ResponseCache (Populate/Lookup) / Cache (Populate/Lookup/Invalidate)	Extension / performance	Caches idempotent GET responses (or any expensive lookup) for a TTL, cutting backend load and latency; a cache key is usually built from path + query params (+ sometimes a header).
PopulateCache / LookupCache / InvalidateCache (KVM-backed)	Extension	Generic cache used for things like a validated JWT's public key, an introspection result, or a computed value — not just HTTP responses.
AccessEntity	Extension	Retrieves an Apigee entity (a Developer, App, or API Product) into flow variables — e.g. to check a Developer's status ("active"/"inactive") or an App's custom attributes before allowing the call through.
RaiseFault	Extension / error handling	Deliberately stops the flow and returns a custom error response (status code, body, headers) — the standard way to enforce every security/quota/validation decision above.
ExtractVariables	Message transformation	Parses values out of the request/response (headers, JSON/XML body via JSONPath/XPath, URI path params) into named flow variables for later policies/conditions to use.

AssignMessage	Message transformation	Builds or modifies a request/response message — add/remove headers, set payload, copy values — commonly used for CORS headers or backend request shaping.
KeyValueMapOperations (KVM)	Extension	Persistent key-value config store scoped to org/env/proxy — used for things like feature flags, backend URLs per environment, or small reference data, without a redeploy.
ServiceCallout	Extension	Makes an out-of-band HTTP call to another service mid-flow — e.g. token introspection, a fraud-check service, or enrichment lookup — before continuing the main flow.
JavaScript / JavaCallout	Extension	Custom logic that no out-of-the-box policy covers — e.g. complex conditional business rules or custom signature schemes.

Q: What's the difference between SpikeArrest and Quota?

A: SpikeArrest smooths short bursts (a rate, e.g. requests/second) to protect infrastructure from sudden spikes regardless of plan; Quota enforces a business/contractual limit (a count over a longer window like a day or month) per app or product. You typically use both together — SpikeArrest first (cheap, protects everyone), then Quota (identity-aware, enforces the plan).

Q: Give a real use case for AccessEntity.

A: Before letting a request through, I look up the calling Developer's Apigee profile via AccessEntity to check a custom attribute like `accountStatus=suspended` — if suspended, I RaiseFault with a 403, even though their token is technically still valid. This lets business/account-status rules live independently of token expiry.

10. Security Deep Dive: OAuth 2.0 Grant Types & MTLS

10.1 OAuth 2.0 grant types — quick recall

Covered in depth in Section 6; the summary table there is the fast-recall version. The one nuance worth adding: **never use the password grant for third-party clients** — it exposes raw credentials to the client app, which is exactly what OAuth was designed to avoid. It's acceptable only for a first-party legacy migration path.

10.2 Mutual TLS (MTLS) — two distinct places it shows up

MTLS means both sides of a TLS connection present and verify a certificate (not just the server, as in normal HTTPS). In this architecture it can apply in two independent places — interviewers often want you to distinguish them:

Where	Who authenticates whom	Apigee configuration	Why
Client ↔ Apigee (inbound)	Apigee verifies the calling client's certificate	A Truststore on the ProxyEndpoint's HTTPS listener referencing the trusted CA(s); the listener is configured to require client certs.	Strong machine-identity for high-trust partner integrations (e.g. bank-to-bank), stronger than API keys alone.
Apigee ↔ backend (outbound)	The AWS-side ALB/NLB verifies Apigee's certificate (and vice versa)	A Keystore on the TargetEndpoint's SSLInfo block holding Apigee's client certificate/private key; the ALB/NLB listener requires and validates it.	Ensures only Apigee (not some other caller who found the ALB's DNS name) can reach the backend directly — this is what makes the AWS-side services safely "internal."

Practically: a Keystore/Truststore in Apigee is an alias holding certificates/keys uploaded to the environment. For outbound MTLS to AWS, you'd generate a key pair, get it signed (or self-signed for internal use), upload the private key + cert as an Apigee Keystore, upload the CA cert AWS should trust as a Truststore reference on the NLB/ALB side (often via AWS Certificate Manager Private CA or a self-managed CA imported into ACM), and reference the Keystore alias in the TargetEndpoint's `<SSLInfo>` block with `<ClientAuthEnabled>true</ClientAuthEnabled>`.

Q: Why would you use MTLS instead of just an API key or OAuth token for backend calls?

A: API keys and OAuth tokens authenticate the *caller's identity/permissions* at the application layer; MTLS authenticates the *transport connection itself* at the network layer, so even if a token leaked, an attacker without the matching private key can't open a TLS session to the backend at all. In regulated flows (payments, PII) we often use both — OAuth for authorization, MTLS for an additional network-level trust boundary.

11. AWS Services Reference (Mapped to the JD)

Every AWS service the JD lists, with its role specifically in this Apigee-fronted architecture. Memorize the "role here" column — that's what you'll actually be asked to connect back to the diagram.

Service	Category	Role in this architecture
EC2	Compute	Hosts the legacy Payments microservice in an Auto Scaling Group behind the ALB.
S3	Storage	Stores Swagger/OpenAPI spec files, CloudFormation templates, Apigee proxy bundle artifacts, and access logs.
VPC	Networking	The private network boundary containing every AWS resource; public subnet holds the ALB, private subnets hold compute + data.
Route 53	Networking / DNS	Resolves the hostname Apigee's TargetEndpoint calls to the current ALB; supports failover routing across regions/AZs.
RDS	Database	Relational store for Orders/Customers data, accessed by the ECS-hosted Orders service.
CloudFormation	IaC / Deployment	Codifies and deploys the entire AWS stack (VPC, ALB, ECS services, RDS, IAM roles, WAF rules) so environments are reproducible and reviewable in Git.
DynamoDB (NoSQL)	Database	Low-latency store for session/preference data used by the Notifications service.
Lambda	Compute (serverless)	Runs the Notifications microservice and/or lightweight SNS/SQS event consumers.
Logging / CloudWatch	Observability	Aggregates logs and metrics from ALB, ECS, Lambda, RDS; backs SLO dashboards and paging alarms.
IAM	Security	Least-privilege roles for ECS tasks, Lambda execution, and any AWS-side automation (CI/CD deploy role, CloudFormation execution role).
Certificate Manager (ACM)	Security	Issues/renews the TLS certificate presented by the ALB (and CloudFront, if used) that Apigee's outbound call trusts.
ELB	Networking	The Application Load Balancer that is AWS's single entry point from Apigee, fanning out to target groups per microservice.
EBS	Storage	Block storage backing the EC2 instances running the Payments service.
ECS	Compute (containers)	Runs the containerized Orders microservice (Fargate = no host management).
CloudFront / WAF	Edge / Security	CDN caching (if static/API responses benefit) plus a Web ACL filtering malicious traffic before it reaches the ALB.

SQS	Messaging	Buffers the async work fanned out from SNS (e.g. "send confirmation email") so consumers can process at their own pace.
SNS	Messaging	Publishes domain events (order.created) once, fanning out to every interested subscriber (SQS, Lambda, email).
SES	Messaging	Sends the actual transactional email/SMS once a consumer picks the event off the queue.

12. Zero-Downtime Deployment & Schema Refactoring

12.1 Zero downtime for Apigee proxies

- **Revision-based deploys.** Apigee deploys a new proxy revision alongside the currently-serving one, then atomically switches traffic — in-flight requests on the old revision complete before it's undeployed (a brief overlap window, not a hard cutover).
- **Backward-compatible changes only on a shared base path.** Additive changes (new optional field, new endpoint) ship without a version bump; breaking changes get a new API version (`/v2/...`) so existing clients on `/v1` are unaffected — this is also the direct answer to "suggest teams on refactoring schemas that are acceptable by all apps."
- **Target server / load-balancing config for canary rollout.** Apigee's TargetEndpoint can define multiple named target servers with weights (e.g. 90% to v1 servers, 10% to v2 servers) so a new backend version is proven under real traffic before a full cutover.

12.2 Zero downtime for the AWS backend

- **Rolling deploys (ECS).** ECS replaces old tasks with new ones a few at a time, only after the new task passes its health check — the ALB never routes to an unhealthy target.
- **Blue/green (CodeDeploy, or two target groups).** Stand up a full "green" fleet, run smoke tests, then shift the ALB listener to green all at once; instant rollback by pointing back at "blue."
- **Database migrations that don't break the running version.** Add-only migrations (new nullable column) deploy before the code that uses them; destructive migrations (drop column) deploy only after all code referencing the old column is retired — the classic "expand/contract" pattern.

12.3 Suggesting schema refactors "acceptable by all apps"

When multiple client apps share one API, you cannot force a synchronized upgrade. The safe pattern: (1) add the new field/shape alongside the old one, (2) get every consumer migrated to the new shape at their own pace, (3) only once analytics show zero traffic using the old shape, deprecate and remove it — often gated with an Apigee `Quota` /header-based deprecation warning policy that flags calls still using the old contract.

13. CI/CD Tooling: Git, Bitbucket, Jira, Jenkins, Sonar, Splunk, Maven

Tool	Role in an Apigee/AWS delivery pipeline
Git / Bitbucket	Source control for Apigee proxy bundles (XML/policies), CloudFormation templates, and microservice code; branching strategy (feature branch → PR → main) gates every change.
Jira	Tracks work items — API change requests, incidents, refactor tasks — usually linked from the PR/commit for traceability.
Jenkins	CI/CD orchestrator: on merge, builds the proxy bundle, runs tests, deploys via Apigee Deploy Maven plugin / apigeecli / Apigee Management API, and separately applies CloudFormation stacks and deploys ECS/Lambda artifacts.
Maven	Build tool — for Apigee, the <code>apigee-deploy-maven-plugin</code> packages and deploys proxy bundles per environment; also builds Java-based microservices/Lambda JARs.
SonarQube (Sonar)	Static code analysis gate in the pipeline — blocks merges/deploys on code smells, security hotspots, or coverage regressions in microservice code.
Splunk	Centralized log analytics — often ingests both Apigee Analytics/Message Logging output and AWS CloudWatch logs, giving one search surface across the whole stack for incident investigation.

A typical pipeline: `Git push` → `Jenkins build` → `Sonar quality gate` → `Maven package/deploy Apigee proxy to env` → `CloudFormation apply (AWS infra)` → `smoke tests` → `promote to next env`. Jira tickets track the change; Splunk (fed by CloudWatch + Apigee Analytics) is where you go first when something breaks in production.

14. SLO / SLA / SLI, On-Call & Blameless Postmortems

Term	Definition	Example in this architecture
SLI (Service Level Indicator)	A measured metric of behavior.	p99 latency of the Orders proxy; % of 5xx responses over 5 minutes.
SLO (Service Level Objective)	An internal target for an SLI.	"p99 latency < 400ms" and "error rate < 0.1%" measured via Apigee Analytics + CloudWatch.
SLA (Service Level Agreement)	An external, often contractual, commitment — usually looser than the SLO, with consequences for breach.	"99.9% monthly uptime" promised to a partner, backed internally by a stricter 99.95% SLO with buffer.

Meeting these in practice means: SpikeArrest/Quota protect the SLO from abusive traffic; CloudWatch alarms fire before the SLO is breached (not after); and ResponseCache + right-sized ECS/Lambda concurrency keep latency within target under load.

On-call & blameless postmortems

- **On-call/on-rotation:** a rotating engineer owns first response to pages (CloudWatch alarm → SNS → PagerDuty/Slack) within an agreed response time, following a runbook for common failure signatures (backend 5xx spike, Apigee quota misconfiguration, cert expiry).
- **Blameless postmortem:** after any priority incident, write up the timeline, root cause, and contributing factors without assigning individual fault — the goal is fixing the system (add a missing alarm, add a canary check, add a runbook step), not finding someone to blame. This is a strong culture signal interviewers listen for explicitly.
- **Permanent closure:** an incident isn't "closed" when service is restored — it's closed when the postmortem's action items (the actual fix, not just the mitigation) are shipped.

15. Real-World Scenarios

These are written the way an interviewer would probe them — situation, what you'd actually do, and why. Practice narrating each in under 90 seconds.

Scenario 1 — Onboarding a brand-new API from a Swagger file

Situation: A product team hands you an OpenAPI/Swagger spec for a new "Inventory" service that will run on ECS in AWS, and asks you to expose it through Apigee.

Approach: Review the spec for REST best-practice issues first (consistent resource naming, correct verb usage, versioned base path, defined error schema) and flag anything that needs the team to fix before you proxy it — this is literally "strong in reviewing specification files" from the JD. Then: create the ProxyEndpoint at `/v1/inventory`, add PreFlow (SpikeArrest → OAuthV2 VerifyAccessToken → Quota), define the TargetEndpoint pointing at the new service's Route 53 hostname behind its own ALB target group, add a RouteRule if it shares a proxy with other services, configure ResponseCache for the read-heavy GET endpoints, write a Postman/automated test collection, and deploy to dev → stage → prod via the Maven/Jenkins pipeline behind a CloudFormation-provisioned AWS stack.

Why it matters: Shows you treat the spec review as a gate, not a rubber stamp, and that you understand the full path from spec to production deploy.

Scenario 2 — Token expires mid-session for a mobile app

Situation: Customers report being logged out unexpectedly while using the mobile app.

Approach: First check whether the app is using the refresh_token flow at all — if the access token's `expires_in` is 3600s and the app has no silent-refresh logic, it will feel like random logouts. Fix: app should proactively call `grant_type=refresh_token` a little before expiry (or handle a 401 by refreshing once, then retrying the original call transparently). On the Apigee side, verify the OAuthV2 policy issues a refresh token for this app's grant type and that its lifetime is sensible (e.g. 30 days) with rotation on use.

Why it matters: Distinguishes an infrastructure bug from a client-integration bug — a very common real production issue.

Scenario 3 — Zero-downtime TLS certificate rotation

Situation: The ALB's ACM certificate is expiring in a week and must be rotated with no client-visible downtime.

Approach: ACM auto-renews certificates it manages (no action needed if DNS validation is already in place) — the interview-worthy nuance is verifying the renewal actually completed and the ALB picked up the new cert (ACM certs are referenced by ARN, and the ALB listener re-fetches transparently — no redeploy required). For a certificate Apigee itself presents (client-facing TLS) or a Keystore used for outbound MTLS to AWS, you'd upload the new cert/key to a new Keystore alias, update the reference, and deploy — Apigee's revision-based deploy model means the old and new certs briefly coexist, so in-flight TLS sessions on the old cert complete normally.

Why it matters: Confirms you understand where each certificate "lives" and that rotation ≠ downtime if you use the platform's built-in overlap mechanisms instead of a hard cutover.

Scenario 4 — Sudden traffic spike from one partner threatens to take down a shared backend

Situation: One partner integration starts sending 50x its normal call volume, and the Payments backend on EC2 begins timing out for everyone.

Approach: Immediately verify/tighten SpikeArrest (protects the whole backend regardless of who's calling) and confirm Quota is correctly scoped per-app (not globally) so this one partner's App is throttled with a 429 without affecting other callers. Medium-term: confirm the EC2 Auto Scaling Group's target-tracking policy and headroom are adequate, and check CloudWatch alarms fired before customers noticed. Longer-term: discuss a rate-limit renegotiation with the partner and consider isolating high-volume partners onto a dedicated target group/environment.

Why it matters: Shows you reach for the cheap, immediate Apigee-side control (SpikeArrest/Quota) before an infrastructure-level fix, and that you think about blast-radius isolation between tenants.

Scenario 5 — Securing Apigee-to-backend calls with mTLS on AWS

Situation: Security asks that the Payments ALB only accept connections from Apigee, not from anything else that discovers its DNS name.

Approach: Configure the TargetEndpoint's SSLInfo with `ClientAuthEnabled=true` referencing an Apigee Keystore holding a client certificate; on the AWS side, configure the NLB (TCP passthrough, so the backend/ALB can do TLS termination and client-cert verification) or the ALB with mutual TLS listener support to require and validate that client certificate against a trusted CA imported into ACM Private CA. Combine with a security group on the ALB/NLB limiting inbound IPs to Apigee's known egress ranges as defense-in-depth (not a replacement for mTLS).

Why it matters: This is the JD's "strong in security like OAuth and mTLS" bullet, applied concretely to the AWS side of the architecture.

Scenario 6 — Priority incident: sudden spike in 5xx errors from the Orders API

Situation: CloudWatch/Apigee Analytics alarms page you at 2am: Orders API error rate jumped from 0.1% to 15%.

Approach: Triage in order: (1) check Apigee Analytics/Splunk for where the errors originate — Apigee fault (bad policy/config, deploy just went out?) vs pass-through from AWS; (2) if pass-through, check ECS service health/target group health checks — are tasks crash-looping, deploy rollback needed?; (3) check RDS — connection pool exhaustion or a slow query is a very common root cause; (4) mitigate first (roll back the last deploy, or scale up), then investigate root cause once customer impact stops. Afterward: write a blameless postmortem — timeline, root cause, contributing factors (e.g., "no alarm existed on RDS connection pool saturation"), and concrete action items with owners and due dates. Close the incident only once those action items ship.

Why it matters: Directly answers "troubleshoot priority incidents, facilitate blameless postmortems and ensure permanent closure of incidents" from the JD — mitigate-first, root-cause-after, fix-the-system-not-the-person.

16. Mock Interview Q&A Bank

Grouped by JD theme. Cover the answer with your hand, read only the question, and answer out loud before checking.

16.1 Architecture & End-to-End Flow

Q1. Walk me through what happens from the moment a client sends a request to the moment it gets a response.

A: (See Section 5 in full.) Client first gets an OAuth token from the token endpoint; calls the Apigee proxy with the bearer token; Apigee's PreFlow runs SpikeArrest → token validation → Quota; if any check fails, RaiseFault returns a 4xx immediately; otherwise a conditional RouteRule picks the TargetEndpoint, which calls into AWS via Route 53/WAF/ALB to the right microservice (ECS/Lambda/EC2); the microservice touches RDS/DynamoDB and optionally SNS/SQS/SES; the response returns the same path, optionally cached by Apigee, back to the client.

Q2. Where can this flow terminate early, and why is that a good thing?

A: At any failed PreFlow check (invalid token, exceeded quota, spike-arrested), Apigee's RaiseFault stops the flow before AWS is ever called. It's good because it protects backend capacity and cost from unauthenticated or abusive traffic — the AWS infrastructure never even sees a request that was never going to be allowed through.

Q3. What's the difference between the ProxyEndpoint and the TargetEndpoint?

A: ProxyEndpoint is the client-facing contract — the base path, PreFlow/PostFlow policies, and TLS listener the client calls. TargetEndpoint is the backend-facing side — the HTTPTargetConnection that calls out to the actual AWS service, plus any target-specific policies (like outbound MTLS SSLInfo) and load balancing across target servers.

Q4. Why would you put a load balancer (ALB) between Apigee and the microservices instead of pointing Apigee directly at instances?

A: Instances/tasks come and go constantly (auto scaling, deploys, failures); the ALB gives Apigee one stable DNS name via Route 53 while it handles health checks and routing to whatever's currently healthy. It decouples Apigee's configuration from AWS's ever-changing compute topology.

Q5. How would you expose three different microservices through a single Apigee proxy base path?

A: Conditional RouteRules keyed on path suffix, header, or API version, each pointing at its own TargetEndpoint/HTTPTargetConnection — one per microservice's AWS hostname (see Section 8).

16.2 Login, Token & Security

Q6. How does a customer log in and get a token in this system?

A: For a real user, via the OAuth 2.0 authorization_code flow (with PKCE): redirect to the IdP's login page, user authenticates there directly, IdP returns a short-lived code, the app exchanges it at Apigee's/IdP's token endpoint for an access token, refresh token, and often an id_token. For system-to-system callers, client_credentials is used instead — no user, just client_id/secret.

Q7. How does Apigee validate the token on every API call?

A: If it's a JWT, Apigee validates it locally — signature against a cached public key/JWKS, expiry, issuer, audience, and required scopes — with no network call needed. If it's an opaque token, Apigee calls the IdP's introspection endpoint via ServiceCallout instead. Any failure raises a fault before the request is routed anywhere.

Q8. What's the trade-off between local JWT validation and introspection?

A: Local validation is fast and doesn't load the IdP, but can't see an instantly-revoked token until it naturally expires. Introspection is authoritative and supports instant revocation, but adds a network hop and IdP load on every request. Many systems mitigate the JWT downside with short expiries (e.g. 15-60 min) plus a deny-list for the rare instant-revoke case.

Q9. Explain MTLS and where you'd use it in this architecture.

A: MTLS means both client and server present certificates, not just the server. Here it applies in two places: optionally between client and Apigee (verifying trusted partner client certs on the inbound listener), and between Apigee and the AWS backend (Apigee presents a client cert the ALB/NLB validates), which is what actually makes the backend "private" even though its ALB technically has a resolvable DNS name.

Q10. Why not just rely on an API key for everything?

A: API keys identify an app, not a user, and are static secrets with no built-in expiry or scope model — if leaked, they're valid until manually revoked. OAuth tokens are short-lived, scoped, and (for user flows) tied to actual user consent, which is a much stronger security posture for anything beyond simple app identification.

Q11. How do you handle IP allow-listing / IP security setup in Apigee?

A: Via an AccessControl policy (or a condition + RaiseFault combo) checking the caller's IP against an allow/deny list, typically combined with security-group-level restrictions on the AWS side as defense-in-depth — Apigee-level for API-contract enforcement, AWS security groups for network-level enforcement.

16.3 Apigee Policies

Q12. When would you use ResponseCache versus a generic Cache policy?

A: ResponseCache is purpose-built for caching whole HTTP responses (great for idempotent, cacheable GETs). The generic PopulateCache/LookupCache pair is more flexible — used for caching intermediate values like a validated JWT's public key, an introspection result, or any computed lookup you don't want to redo on every request.

Q13. What does the RaiseFault policy actually do, mechanically?

A: It stops normal flow execution and returns a custom response (status code, headers, body) you define — it's how every security/quota/validation check in Apigee actually communicates failure back to the client, usually triggered from a fault rule tied to a condition (e.g. a failed VerifyAccessToken).

Q14. What's AccessEntity used for, concretely?

A: Retrieving Apigee-managed entities (Developer, App, API Product) into flow variables mid-request — e.g., checking a Developer's custom "accountStatus" attribute to allow a business rule ("suspended accounts get a 403") that's independent of whether their token is still technically valid.

Q15. How would you enforce per-partner rate limits without redeploying the proxy for every new partner?

A: Model the limit as an API Product/App attribute in Apigee (or a KVM entry) rather than a hard-coded value in policy XML, and reference that attribute in the Quota policy's dynamic configuration — new partners just get configured via the Apigee entity model, no code/proxy change needed.

Q16. What's KeyValueCollectionOperations useful for?

A: Storing small, environment-scoped config (feature flags, backend URLs per environment, reference data) that you want to change without a full proxy redeploy — read via LookupCache/KVM policy at runtime.

16.4 AWS Services & Networking

Q17. What's the difference between an ALB and an NLB, and when would you choose each?

A: ALB operates at layer 7 (HTTP/HTTPS) and supports host/path-based routing, ideal for microservice APIs. NLB operates at layer 4 (TCP/UDP), ultra-low latency, and is the right choice when you need raw TCP passthrough — for example, if the backend itself needs to terminate MTLS and inspect the client certificate rather than having the load balancer do it.

Q18. What role does Route 53 play between Apigee and AWS?

A: It resolves the hostname configured in Apigee's TargetEndpoint to the current ALB/NLB endpoint; it can also do health-check-based failover so that hostname keeps resolving to a healthy region/AZ without Apigee configuration ever changing.

Q19. Why put a VPC with public/private subnets in front of the microservices?

A: Only the ALB sits in a public subnet, internet/Apigee-reachable; compute (ECS/EC2/Lambda-in-VPC) and data (RDS/DynamoDB endpoints) live in private subnets with no direct inbound route — network isolation as a baseline security control, independent of application-layer auth.

Q20. When would you choose ECS Fargate versus EC2 for a microservice?

A: Fargate when you want to run containers without managing hosts — patching, capacity planning and OS maintenance are AWS's problem, at a slightly higher per-unit cost. EC2 (with your own Auto Scaling Group) when you need more control (custom AMIs, specific instance types/GPUs, cost optimization at scale) or you're running a legacy workload not yet containerized.

Q21. When would you use Lambda instead of ECS for a microservice?

A: For event-driven, spiky, or low/no-traffic workloads (notifications, glue code, async consumers) where scale-to-zero and pay-per-invocation matter; ECS/EC2 remain a better fit for steady, latency-sensitive, long-running services with predictable load.

Q22. What's DynamoDB good for here that RDS isn't?

A: Single-digit-millisecond key-value lookups at scale with flexible schema — good for session state, preferences, or idempotency keys. RDS is better for data with real relational structure and the need for joins/transactions across entities, like Orders-to-Customers.

Q23. Explain how SNS and SQS work together in an event-driven flow.

A: SNS is publish once, deliver to many subscribers (fan-out) — e.g. an order.created event. SQS is a durable queue a single consumer (or consumer group) pulls from at its own pace. Combining them (SNS topic fanning out to multiple SQS queues) lets several independent consumers each reliably process the same event without one slow consumer blocking another.

Q24. What's WAF protecting against that Apigee's own policies don't already cover?

A: WAF operates on raw HTTP traffic patterns (SQLi/XSS signatures, known bad IPs, generic rate-based rules) before the request even reaches Apigee's more application-aware policies; it's a broad, signature-based first line of defense, complementary to — not a replacement for — Apigee's identity/quota/business-rule enforcement.

Q25. What does IAM actually secure in this picture?

A: Which AWS principal (an ECS task role, a Lambda execution role, a CI/CD deploy role) can call which AWS API — e.g. the Orders ECS task's role only allows read/write to its specific RDS instance and nothing else, following least privilege.

Q26. What's the role of CloudFormation in the delivery pipeline?

A: It's the infrastructure-as-code layer — VPC, ALB, target groups, ECS services, RDS instances, IAM roles and WAF rules are all defined declaratively in templates checked into Git, and applied via the CI/CD pipeline so every environment (dev/stage/prod) is provisioned identically and changes are code-reviewed.

16.5 Zero Downtime, CI/CD & Operations

Q27. How do you achieve zero downtime when deploying a new Apigee proxy revision?

A: Apigee deploys the new revision alongside the current one and switches traffic without a hard cutover — in-flight requests on the old revision finish normally, new requests go to the new revision, so there's an overlap window rather than a gap.

Q28. How do you achieve zero downtime on the AWS backend during a deploy?

A: Rolling deploys in ECS (replace tasks a few at a time, gated by health checks) for routine releases, or blue/green (two target groups, instant listener cutover, instant rollback) for higher-risk releases.

Q29. A schema change is needed but multiple client apps depend on the current shape — how do you refactor safely?

A: Expand/contract: add the new field/shape alongside the old one (never remove first), let every consumer migrate at their own pace (tracked via Apigee Analytics showing who's still using the old shape), and only remove the old field once usage drops to zero.

Q30. Describe your ideal CI/CD pipeline for an Apigee + AWS change.

A: Git/Bitbucket feature branch → PR (Jira-linked) → Jenkins build → SonarQube quality gate → Maven packages/deploy the Apigee proxy bundle to the target environment (apigee-deploy-maven-plugin) → CloudFormation applies any AWS infra changes → automated smoke tests → promote through dev/stage/prod. Splunk (fed by CloudWatch + Apigee Analytics) is the first place to look if anything regresses post-deploy.

Q31. What's the difference between SLI, SLO and SLA?

A: SLI is the measured metric (e.g. p99 latency). SLO is the internal target for that metric (e.g. "<400ms"). SLA is the external, often contractual, commitment to a customer/partner — typically set looser than the internal SLO so there's buffer before a real breach.

Q32. Tell me about how you'd handle being on-call for this system.

A: Follow the rotation's response-time SLA, triage using a runbook (Apigee fault vs AWS-side error vs data-layer issue), mitigate customer impact first (rollback/scale), investigate root cause after, and write up a blameless postmortem with concrete action items and owners — closing the incident only once those items ship, not once service is merely restored.

Q33. Why "blameless" postmortems specifically?

A: Blame makes people hide information and stop reporting near-misses, which makes the system less safe over time. Blameless postmortems assume good intent and focus on what let the failure happen — missing alarms, unclear runbooks, risky deploy processes — so the fix is systemic and the next incident is genuinely less likely.

16.6 Behavioral / General (matches "urge to deliver fast," "collaborate asynchronously")

Q34. Tell me about a time you saw something broken and fixed it without being asked.

A: Structure with STAR: Situation (noticed a proxy without SpikeArrest exposing a backend to risk), Task (nobody assigned it, but it was clearly a gap), Action (added the policy, tested it, opened a PR with context), Result (backend protected, became the team's default template going forward). Emphasize initiative + collaboration, not going rogue — you still went through review.

Q35. How do you communicate asynchronously with a distributed team across time zones?

A: Write self-contained updates (context + what changed + what's needed from the reader) in Jira/Slack/PR descriptions rather than relying on live meetings; document decisions where the next person will look for them; over-communicate assumptions so someone eight time zones away isn't blocked waiting for a live answer.

Q36. You're new to AWS but strong in GCP Apigee — how do you ramp up quickly on a live production system?

A: Map unfamiliar AWS concepts to GCP equivalents I already understand deeply (this guide is literally that exercise), read the existing CloudFormation templates as the ground truth for "how this specific system actually uses AWS" rather than learning AWS in the abstract, and pair with a teammate on the first few AWS-side changes before working independently.

Q37. How do you balance "deliver quickly" with not breaking production?

A: Ship small, reversible changes behind the zero-downtime mechanisms already in place (revision overlap, rolling/blue-green deploys, expand/contract schema changes) so "fast" and "safe" aren't in tension — the safety net is what lets you move fast, not something that trades off against it.

17. Final Recall Checklist

Read this once on interview morning. If you can explain every line below out loud, you're ready.

Topic	Can you say it in one breath?
End-to-end flow	Token first (client ↔ IdP), then bearer-token call to Apigee, PreFlow validates (SpikeArrest → token → Quota), RouteRule picks the microservice, egress via Route53/WAF/ALB into AWS, backend touches RDS/DynamoDB/SNS/SQS/SES, response bubbles back through Apigee (cache) to client.
Where flow ends early	RaiseFault on any failed PreFlow check — before AWS is ever called.
Token validation	JWT = local (signature/exp/iss/aud/scope, cached JWKS); opaque = ServiceCallout introspection. Trade-off: speed vs instant revocation.
MTLS, two places	Client ↔ Apigee (Truststore, inbound) and Apigee ↔ AWS backend (Keystore, outbound via TargetEndpoint SSLInfo) — different purpose, different config.
Two layers of routing	Apigee RouteRule picks the microservice; AWS ALB target group picks the healthy instance of it.
Caching, AccessEntity, RaiseFault	ResponseCache/PopulateCache for perf; AccessEntity for Developer/App/Product lookups; RaiseFault as the universal "stop and respond" mechanism.
SpikeArrest vs Quota	Rate smoothing (protects infra, identity-agnostic) vs business volume limit (identity-aware, contractual).
Zero downtime	Apigee revision overlap + AWS rolling/blue-green deploys + expand/contract schema migrations.
SLI / SLO / SLA	Measured metric / internal target / external contractual promise — SLA is looser than SLO by design.
Incident response	Mitigate first, root-cause after, blameless postmortem, close only when the actual fix — not just the mitigation — ships.
AWS services, one line each	See Section 11 — know the "role here," not just the definition.
CI/CD chain	Git/Bitbucket → Jira-linked PR → Jenkins → Sonar gate → Maven deploys Apigee bundle → CloudFormation applies AWS infra → Splunk (CloudWatch + Apigee Analytics) for post-deploy visibility.

If you get stuck on a pure-AWS trivia question: answer with the GCP analogy first, then reason forward — interviewers are testing engineering judgment and how fast you learn, not whether you already memorized every AWS service. This entire guide was built on exactly that principle.

Good luck — you've got the hardest part (deep Apigee fluency) already. This is the AWS vocabulary layered on top of judgment you already have.